

Workshop: Implementing a Class with Constructors, Destructors, and Dynamic Memory Allocation

In this workshop, you will design and implement an `Employee` class that uses dynamic memory to store an employee name. You will practice writing constructors, a copy constructor, and a destructor that correctly manage resources.

Part 1 – Design the `Employee` Class

Create a header file called `Employee.h` and declare a class named `Employee`.

Requirements

Your `Employee` class must have the following **private** member variables:

- An integer to store the employee identification number.
- A `float` to store the employee salary, in CAD.
- A pointer to `char` that will store the employee name as a **dynamically allocated** C-style string.

Declare the following member functions in the class:

1. **Base (default) constructor**

- Takes no parameters.
- Initializes the identification number to `0`.
- Initializes the salary to `0.0f` (0.0 CAD).
- Initializes the name pointer to `nullptr`.

2. **Overloaded constructor**

- Takes three parameters: an `int` (id), a `float` (salary), and a `const char*` (name).
- Stores the provided identification number and salary.
- Dynamically allocates enough memory to store the provided name (including the null terminator) and copies it into the internal `char*` member.

3. **Copy constructor**

- Takes a `const Employee&` as a parameter.
- Creates a **deep copy** of the source object, including allocating new memory for the name and copying its contents, instead of just copying the pointer.

4. **Destructor**

- Deallocates any dynamically allocated memory used to store the name.
- Ensures that there are no memory leaks.

Make sure that all member functions are only **declared** in the header file at this stage.

Part 2 – Implement the `Employee` Class

Create a source file called `Employee.cpp` and provide the implementations for all member functions declared in `Employee.h`.

Implementation Tasks

1. Base constructor implementation

- Set the identification number to `0`.
- Set the salary to `0.0f`.
- Set the name pointer to `nullptr`.

2. Overloaded constructor implementation

- Initialize all member variables (you may use a member initializer list).
- Allocate memory for the name using `new[]`.
- Copy the characters from the incoming `const char*` parameter into the newly allocated array.
- Ensure that the string stored in the object is properly null-terminated.

3. Copy constructor implementation

- Copy the identification number and salary from the source object.
- If the source object has a non-null name pointer, allocate new memory for the name in the new object and copy the characters over.
- If the source object has a `nullptr` name, set the new object's name pointer to `nullptr` as well.

4. Destructor implementation

- Use `delete[]` to deallocate the dynamically allocated name array (if the pointer is not `nullptr`).
- Set the name pointer to `nullptr` after deletion (optional, but good practice).

Double-check that your implementations do **not** use the C++ `std::string` class and that all name handling is done via C-style strings (`char*`).

Part 3 – Test and Debug the `Employee` Class

Use the provided `main.cpp` file to test your `Employee` class. The goal is to verify that your constructors, copy constructor, and destructor behave correctly and that no memory leaks occur.

Make sure to run your code through a debugger and check that constructors, copy constructors, and destructors are called when expected.

Testing Tasks

1. Test the base constructor

- Check that a base constructor is called when line 10 of `main.cpp` is executed.

2. Test the overloaded constructor

- Check that the overloaded constructor is called when line 13 of `main.cpp` is executed.

3. Test the copy constructor

- Check that the copy constructor is created when lines 16 and 19 are executed.

4. Check for destructor

- Check that the destructor is called when the function `showEmployee` returns.
-

Deliverables

Submit the following files:

- `Employee.h`
- `Employee.cpp`
- `main.cpp` (which is provided)

Your final code must:

- Compile and run without errors or memory leaks.
- Correctly implement the base constructor, overloaded constructor, copy constructor, and destructor.
- Use dynamic memory (`new[]` / `delete[]`) for storing the employee name and ensure that a deep copy is performed in the copy constructor.